

Assignment 4

Files Manipulation

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

23.06.2024

Assignment

Disk Scheduling Algorithms

You are developing a driver for a disk controller. Implement the following disk scheduling algorithms:

```
// First Come, First Served
int FCFS(struct device_t* device, int requests[], int size);
// Shortest Seek First
int SSF(struct device_t* device, int requests[], int size);
// Elevator algorithm
int SCAN(struct device_t* device, int requests[], int size);
```

The requested cylinders are located in the array `requests` of size `size`. For each algorithm, you have to process all the cylinder blocks that are located in `requests`.

Since you are not expected to manually access the fields of `struct device_t`, you **must** use the following functions to interact with the device disk controller:

```

// Move the arm of the disk to the provided cylinder and return the new position
int dev_move_to_cylinder(struct device_t *device, int cylinder);

// Get the current cylinder pointed by the arm
int dev_get_cylinder(const struct device_t *device);

// Get the maximum of cylinders on the drive
int dev_get_max_cylinder(const struct device_t *device);

// Process the current cylinder (only call if there is a request for it)
void dev_work(struct device_t *device);

// Get the current direction of the arm (can be either LEFT or RIGHT)
enum direction dev_get_direction(const struct device_t *device);

// Change the direction of the arm
void dev_switch_direction(struct device_t *device);

// Additionally, here is the definition of enum direction
enum direction {
    LEFT, // Going toward lower cylinder number
    RIGHT, // Going toward higher cylinder number
};

```

These functions are used to interact with the disk. For instance if you need to process a request for the cylinder number 12, you would use the functions like this:

```

...
dev_move_to_cylinder(device, 12);
dev_work(device);
...

```

Finally, all of the functions should return the total seek time to process the requests. The seek time is calculated based on the total distance traveled by the arm.

The tests will check that you process all the requests in the right order and that the returned seek time is correct.

Note

The strategy for SSF, in case of two cylinders being equidistant to the current arm position, is to select the one following the current direction of the arm.

The direction functions are helpful to give you the initial arm direction. Note that we are not checking that the arm position is consistent afterward.

Interrupt Service Routine

You are given the following interrupt service routine. Its role is to copy the buffer from a network interface controller (NIC) to the kernel, then to process this buffer later in another function (using `schedule_upper_half`).

```

#define SIZE 4096 // represents the buffer size in bytes
uint8_t **const reg_buffer_device;

uint8_t buffer[SIZE];

int ISR() {
    uint8_t *dev_buffer = *reg_buffer_device;
    ack_reg_copied();

    copy_dev_buffer(dev_buffer, buffer, SIZE);

    ack_buffer_copied();

    schedule_upper_half(buffer);
    return 0;
}

```

Masking/unmasking the interrupts can be achieved with these two functions:

```

disable_interrupts();
enable_interrupts();

```

The global variable `uint8_t **const reg_buffer_device` is a constant pointer to a device register that contains an address to one of its internal buffer of size defined by `SIZE`. The register value can change depending on the status of the NIC. We suppose that this address and the register address are always valid.

Tasks

There are mistakes in this code, it's causing problem when multiple interruptions are triggered in a short amount of time.

Modify the code to make it valid with the following constraints:

1. The code must be made re-entrant
2. `copy_dev_buffer` can't be run while interruptions are masked because it would make the system unresponsive